

Подготовка к экзамену по C++ (ИТМО)

Тема 0: Встроенные типы, числа в памяти и строки

Hardcore Theory Edition

14 июня 2026 г.

1 Встроенные типы данных и их размеры

1.1 Теория: Размеры и гарантии стандарта

Стандарт C++ **не фиксирует** жестко размер большинства типов в байтах. Он фиксирует лишь их соотношение.

- `char` — гарантированно 1 байт (минимальная адресуемая ячейка памяти).
- Соотношение: $1 \equiv \text{sizeof}(\text{char}) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{longlong})$.
- На современных 64-битных системах (x64) обычно: `int` = 4 байта, `long long` = 8 байт, а вот размер `long` может плавать (в Windows он 4 байта, в Linux — 8 байт).
- **Фиксированные типы (<stdint>):** Если вам нужен точный размер для работы с сетью или битами, всегда используйте `int32_t`, `uint64_t`, `int8_t` и т.д.

Суть на пальцах (Жизненная аналогия)

Стандартные типы (`int`, `long`) — это коробки с надписями "Средняя" и "Большая". В разных магазинах (компиляторах) их размер будет отличаться, главное правило — "Средняя" не может быть больше "Большой". А типы из <stdint> (`int32_t`) — это контейнеры, изготовленные по строгому ГОСТу. Вы точно знаете, что их объем ровно 32 бита на любом заводе в мире.

2 Представление целых чисел

2.1 Дополнительный код (Two's Complement)

Беззнаковые (unsigned): Кодировются как обычные числа в двоичной системе.

Знаковые (signed): В современных архитектурах отрицательные числа представляются в **дополнительном коде**. Как получить $-X$ из X : 1. Инвертируем все биты (0 меняем на 1, 1 на 0). 2. Прибавляем 1.

Пример для 8-битного числа (-5) : $5_{10} = 00000101_2 = 11111010_2 + 1 = -5_{10}$.

2.2 Переполнение (Overflow) — Любимый вопрос на экзамене

- **Unsigned переполнение:** Это *определенное поведение*. Число просто "пойдет по кругу" по модулю 2^N . Максимум $+ 1 = 0$.
- **Signed переполнение:** Это **Undefined Behavior (UB)**. Если `int` превысит свой лимит (например, $2\,147\,483\,647 + 1$), компилятор имеет право сделать с вашей программой всё, что угодно. Оптимизатор может просто удалить ветки кода, полагая, что переполнение невозможно.

Суть на пальцах (Жизненная аналогия)

Беззнаковое переполнение (**unsigned**) — это механический одометр в старой машине. Когда пробег доходит до 999 999, он просто щелкает и сбрасывается в 000 000. Всё ожидаемо. Знаковое переполнение (**signed**) — это если бы при попытке проехать миллионный километр у машины отвалились колеса, а двигатель взорвался (UB).

3 Представление вещественных чисел (IEEE 754)

3.1 Анатомия float и double

Дробные числа (`float` — 32 бита, `double` — 64 бита) не хранятся точно. Они используют научную нотацию (стандарт IEEE 754) и делятся на 3 части:

$$(-1)^{Sign} \times 1.Mantissa \times 2^{Exponent - Bias}$$

1. **Sign (Знак):** 1 бит.
2. **Exponent (Экспонента/Порядок):** 8 бит для `float`. Определяет масштаб (где стоит запятая).
3. **Mantissa (Мантисса):** 23 бита для `float`. Хранит сами значащие цифры числа.

3.2 Проблема точности и ϵ (Эпсилон)

Так как мантисса ограничена, числа, которые нельзя представить как сумму степеней двойки (например, 0.1), становятся **бесконечными дробями** в двоичной системе. Они обрезаются, из-за чего возникает погрешность. **Никогда не сравнивайте float или double через ==!**

```
1 #include <iostream>
2 #include <cmath>
3
4 int main() {
5     double a = 0.1 + 0.2;
6     double b = 0.3;
7
8     // Этот if ВЫПОЛНИТСЯ! a == 0.30000000000000004
9     if (a != b) {
10         std::cout << "0.1 + 0.2 != 0.3 ! Surprise!\n";
11     }
12
13     // Правильное сравнение чисел с плавающей точкой
14     double epsilon = 1e-9;
```

```

15     if (std::abs(a - b) < epsilon) {
16         std::cout << "Numbers are practically equal.\n";
17     }
18     return 0;
19 }

```

4 Строки: C-строки vs std::string и SSO

4.1 C-style строки (Массивы char)

Сырая строка из языка C — это просто указатель на массив байтов (`char*`), который обязательно должен заканчиваться терминальным нулем `'\0'`.

- Чтобы узнать длину, функция `strlen()` бежит по памяти $O(N)$, пока не встретит `'\0'`.
- Очень легко выйти за границы массива и получить Segmentation fault.

4.2 std::string и магия SSO (Small String Optimization)

`std::string` — это умный класс-контейнер, инкапсулирующий работу с памятью. Внутри он хранит размер (`size`), вместимость (`capacity`) и указатель на память.

Вопрос на 5: Всегда ли `std::string` выделяет память в куче (через `new`)? Нет! Вызов кучи — это медленно. Чтобы оптимизировать работу с короткими строками (обычно до 15-22 символов в зависимости от компилятора), применяется **SSO**.

Внутри `std::string` есть небольшой внутренний массив. Если строка короткая, она сохраняется **прямо внутри самого объекта** (на стеке). Если строка становится больше лимита, `std::string` выделяет память в куче и использует свой внутренний массив для хранения указателей на эту кучу (используя `union` под капотом).

Суть на пальцах (Жизненная аналогия)

C-строка — это поезд, у которого нет расписания. Чтобы узнать, сколько в нем вагонов, нужно бежать вдоль состава, пока не упрушься в последний вагон со специальным красным флагом (`'\0'`). `std::string` — это хитрый курьер (SSO). Если посылка маленькая (до 15 букв), он кладет её себе в карманы куртки (быстро, на стеке). Если посылка огромная, он арендует склад (куча), а в карман кладет ключи от склада.

```

1 #include <iostream>
2 #include <string>
3
4 // Демонстрация работы capacity и SSO
5 int main() {
6     std::string s = "Hello";
7     // Строка короткая, скорее всего работает SSO (память в куче не выделена)
8
9     std::cout << "Size: " << s.size() << "\n"; // 5
10    // capacity - сколько символов влезет без реаллокации памяти
11    std::cout << "Capacity: " << s.capacity() << "\n"; // Например, 15
12
13    // Добавляем длинный текст
14    s += " this is a very long string that will force heap allocation";
15

```

```
16     std::cout << "New Size: " << s.size() << "\n";
17     // capacity резко увеличится (выделится кусок памяти в куче)
18     std::cout << "New Capacity: " << s.capacity() << "\n";
19
20     return 0;
21 }
```